IT5403
OPERATING SYSTEMS

PRECISION AND EFFICIENCY: A CASE STUDY ON REAL TIME OPERATING SYSTEMS (RTOS)

Done By

Kabilesh Selvakumar E	2022506078
Yuvaraj K	2022506079
Seshathri A	2022506102
Krishnaa S	2022506104
Abdul Rahman S M	2022506311

DEPARTMENT OF INFORMATION TECHNOLOGY
MADRAS INSTITUTE OF TECHNOLOGY,
ANNA UNIVERSITY, CHENNAI – 600 044

March 2023

TABLE OF CONTENTS

S. No	Title	Page No
1	An Introduction to RTOS	1
2	Types of RTOS	3
3	When is an OS considered Real Time?	5
4	Amazon FreeRTOS™	6
5	FreeRTOS Kernel	7
6	Data Types and Coding Guidelines	8
7	Memory Management	11
8	Task Management	20
9	Scheduling Algorithms	24
10	Queue Management	28
11	Software Timer Management	31
12	Interrupt Management	33
13	Resource Management	39
14	Other Features of FreeRTOS	43
15	Conclusion	48
16	References	50

An Introduction to RTOS

A General Purpose Operating System (GPOS) is one that can manage a high number of processes and provide a high throughput - executions per unit time. The most popular operating systems that we use in our day-to-day lives are all mostly general purpose operating systems. These systems aim to maintain a high throughput, that is, they want to serve and complete as many processes as possible in a given period of time. Consider an example - there is one high-priority task or process and five lower priority ones to be executed. In order to achieve the desired high throughput, the general purpose operating system would delay the execution of this high-priority process to complete the lower priority ones.

These general purpose operating systems use a fairness policy to carry processes and threads to the CPU. Such a policy allows it to give the high output that desktop and server applications require to work efficiently. However, it can't guarantee that a high priority process or time-critical threads will execute before low priority tasks.

So, a general purpose operating system works quite well for devices that perform multiple tasks at the same time. However, there are many, many scenarios where a high-priority task needs to be completed within a required timeframe, and this is where Real-Time Operating System, or RTOS, comes in.

RTOS is a specialised operating system designed explicitly to manage resources, execute programs, and process data for real-time applications. What differentiates an RTOS from other operating systems is its primary focus: fast and predictable responses to events or data inputs.

This concept of real-time computing dates back several decades. The first recognised RTOS was the real-time monitor program developed by Cambridge University in the 1960s. This was ground breaking - you could now allow multiple processes to run concurrently *within strict timing constraints.*

Since then, RTOS has evolved with technological advancements and the growing demand for real-time performance. With each iteration, these systems have become more robust, efficient, and feature-rich, catering to a wide range of industries such as aerospace, defence, medical, science, multimedia applications, and more.

But what exactly does *real-time* mean? In some way or the other, isn't technically anything that we do in any operating system happening in real time? The definition, or rather interpretation, is different when it comes to RTOS.

Real-time means the system adheres to real-time constraints – or operational deadlines established from event inception to system response. These responses might need execution within seconds, milliseconds, or even microseconds, depending on the application's demands.

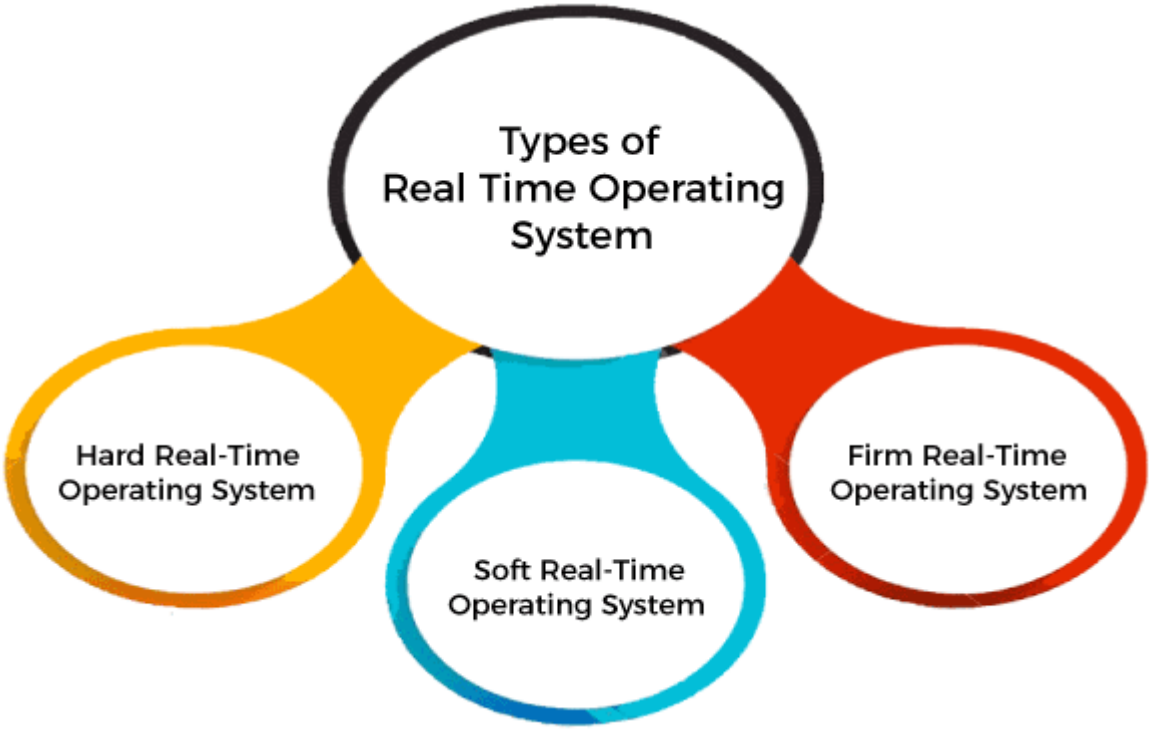
Any RTOS must have a known maximum time for each of the critical operations that it performs, and it should be able to guarantee that maximum most of the time, that is, it must complete these operations within that timeframe. In other words, it is a *time-bound* system.

Consider the following example: you are designing an airbag system for a new model of a car. In this case, a small error in timing that causes the airbag to deploy a little too early or a little too late could be catastrophic and cause injuries. It could very well be the difference between life and death.

Real time operating systems have similar functions as general purpose operating systems like Windows, macOS, and Linux, but are designed in such a way that a scheduler can meet specific deadlines for different tasks. GPOSes are certainly better for consumer use where multiple tasks can be handled concurrently and efficiently, but RTOSes are the way to go when there is a focus on quickness and accuracy.

Types of RTOS

There are three main types of real time operating systems based on their level of real-time responsiveness:



1) Hard RTOS

Hard real-time systems have strict timing requirements. These systems are designed for applications where missing a deadline is considered a system failure. For instance, in an anti-lock braking system, not responding to a sensor reading in time could be catastrophic.

To achieve hard real-time capabilities, RTOS uses specialized scheduling algorithms and techniques. These systems prioritize critical tasks over non-critical ones and employ mechanisms like priority inversion avoidance and deadline monitoring. This ensures that time-critical operations receive the necessary resources and are executed without delays or interruptions.

Examples of industries where hard real-time systems are crucial include aerospace, defence, automotive, and industrial automation. In such applications,

milliseconds matter, whether controlling an aircraft's flight path or managing a production line in a factory. Hard real-time systems are vital in ensuring safety, reliability, and efficiency in these demanding environments.

2) Firm RTOS

These systems have deadlines that must be met most of the time, but occasional missed deadlines may be tolerated.

In firm real-time systems, meeting timing constraints is crucial for most tasks, as it directly impacts system performance and reliability. However, these systems allow limited flexibility in missing some deadlines without catastrophic consequences.

These types of RTOS find applications in various industries where timely responses are critical but not enough to cause severe failures or safety hazards if occasionally missed. Some examples include industrial automation, power plants, traffic control systems, and telecommunications networks.

3) Soft RTOS

Soft real-time systems prioritize responsiveness but with some flexibility in meeting deadlines. They do not have stringent timing requirements and allow occasional delays without compromising the overall functionality.

In soft RTOS, processes have certain time constraints or deadlines to meet, but missing these deadlines does not cause catastrophic consequences. For example, multimedia applications like streaming videos or playing music can tolerate small delays without impacting the user experience significantly.

Soft real-time systems' flexibility allows them to handle applications that can accommodate occasional delays without major issues. This makes them suitable for industries such as gaming, entertainment, and communication networks.

When is an OS considered Real Time?

To be considered *Real Time*, an operating system needs to meet a series of key requirements:

Requirement	Explanation
Determinism	Ensures operations happen within a strictly defined timeframe.
Task Prioritisation	Prioritizes tasks based on their urgency and allocates CPU resources accordingly.
Concurrency	Supports running multiple tasks at the same time or creating the appearance of simultaneity through multitasking.
Memory Management	Efficiently allocates and deallocates memory segments without fragmentation, maximizing available memory at all times.
Minimal Latency	Ensures the time taken to respond to an external event is minimal and consistent.
Reliability and Stability	Highly necessary as many real-time systems are mission-critical.
Footprint	Must have a small memory footprint due to frequent embedding in systems with constrained resources.

Amazon FreeRTOS™

There are many different versions and many different providers of RTOS. One of the most popular ones in 2024 is FreeRTOS.

FreeRTOS is the most adopted RTOS for designs. It is an open-source, MIT licensed operating system targeted at embedded applications that need real-time event processing. It was founded by Richard Barry around 2003, and was further maintained by his company Real Time Engineering Ltd. In 2017, the stewardship of this project was passed to Amazon Web Services.

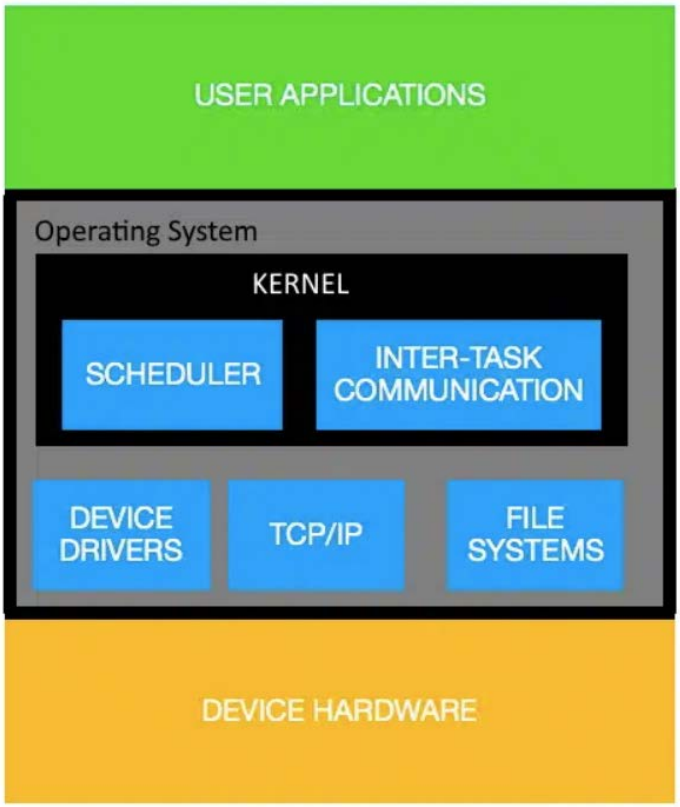
Amazon Web Services provide many cloud hosted services, and a large number of these are being designed for Internet of Things or IoT applications. On the other side of things, there are Microcontroller Unit or MCU devices, which are basically small and self-contained computers. These are being connected to the cloud in order to make them better and more effective. The thing is, there are a lot of security and connectivity components that need to be built in order to make this all work, and since a significant portion of MCU devices already run the FreeRTOS kernel, Amazon chose to use this and provide it with the resources necessary to extend their offering into fully integrated security and connectivity libraries. They are also ensuring that there is room for development and support long into the future. So, with this move, Amazon ensures that the wider FreeRTOS ecosystem is strong, and at the same time continues to improve the web-based IoT services that they provide.

In simpler words, Amazon FreeRTOS makes it easy to program commonly needed IoT capabilities into your device, like configuring them to a local network using common connectivity options like Wi-Fi. It comes with libraries to help secure device data and connections, including support for data encryption and key management. It also includes support for Transport Layer Security (TLS v1.2) to help devices connect securely to the cloud.

These are all what Amazon adds on top of the already existing FreeRTOS platform. What we’d like to take a deeper look at is FreeRTOS itself, and understand the many intricacies within it.

FreeRTOS Kernel

FreeRTOS is a collection of C libraries comprised of a real-time kernel, and a set of modular libraries that implement complementary functionality.



The FreeRTOS kernel is a real-time kernel or real-time scheduler that enables applications built on FreeRTOS to meet their hard real-time requirements. It is ideally suited to deeply embedded real-time applications that run on microcontrollers or small microprocessors. This type of application typically includes a mix of both hard and soft real-time requirements.

FreeRTOS is designed to be small and simple. The kernel itself consists of only three or four C files. To make the code readable, easy to port, and maintainable, it is written mostly in C, but there are a few assembly functions included where needed (mostly in architecture specific scheduler routines). FreeRTOS's code breaks down into three main areas: tasks, communication, and hardware interfacing.

FreeRTOS provides methods for multiple threads or tasks, and semaphores. Thread priorities are supported. There are none of the more advanced features typical found in operating systems like Linux or Microsoft Windows, such as device drivers, advanced memory management, user accounts, and networking. The emphasis is on compactness and speed of execution. FreeRTOS can be thought of as a 'thread library' rather than an 'operating system', although command line interface and POSIX like IO abstraction add-ons are available.

Data Types and Coding Guidelines

FreeRTOS can be built with approximately twenty different compilers and can run on more than forty different processor architectures. Each supported combination of compiler and processor is called a *FreeRTOS port.*

The FreeRTOS kernel can be configured by constants defined in the FreeRTOSConfig.h header file. However, in the source files, FreeRTOS.h is included, which automatically includes FreeRTOSConfig.h at the appropriate time. This configuration header file consists of constants such as configUSE_PREEMPTION, which defines whether FreeRTOS uses co-operative or pre-emptive scheduling.

The OS is supplied as a set of C source files, some of which are common to all the ports, while others are specific to one port or the other. For example, there are the source files tasks.c and list.c, which are necessary source files and are always required. The table below provides information about some other optional source files common to all ports.

Source File	Description
queue.c	Provides both queue and semaphore services. It is nearly always required.
timers.c	Provides software timer functionality
event_groups.c	Provides event group functionality.
stream_buffer.c	Provides both stream buffer and message buffer functionalities.
croutine.c	Implements the FreeRTOS co-routine functionality, which are intended for use on very small microcontrollers.

Data Types

Each port in FreeRTOS has a unique portmacro.h header file that contains many things, the most important of which are definitions for two port-specific data types:

1) TickType_t

Tick Interrupt	Periodic Interrupt configured by FreeRTOS
Tick Count	No. of Tick Interrupts occurred since the start of the application; a measure of time
Tick Period	Time between two tick interrupts
Times	Specified as multiples of tick periods

TickType_t is the data type used to hold the tick count value, and to specify times. It can be an unsigned 16, 32, or 64-bit type.

2) BaseType_t

This is the most efficient data type for the architecture. It is typically a 64-bit type on a 64-bit architecture, a 32-bit type on a 32-bit architecture, and so on. It is generally used for return types that take only a very limited range of values, and also for pdTRUE/pdFALSE type Booleans.

Variable Names

Prefix	Type
c	char
s	int16_t
l	int32_t
x	BaseType_t, and other non-standard types such as structures, task handles, queue handles, etc.
u	Unsigned
p	Pointer

Function Names

Functions are prefixed with both the type they return and the file they are defined within. For example:

Function Name	Return Type	Defined Within
vTaskPrioritySet()	void	tasks.c
xQueueReceive()	BaseType_t	queue.c
pvTimerGetTimerID()	pointer to void	timers.c

Macro Names

Most macros are written in upper case, and are prefixed with lower case letters that indicated where the macro is defined. Given below are a list of prefixes.

Prefix	Location of Macro Definition	Example
port	portable.h or portmacro.h	portMAX_DELAY
task	task.h	taskENTER_CRITICAL()
pd	projdefs.h	pdTRUE
config	FreeRTOSConfig.h	configUSE_PREEMPTION
err	projedefs.h	errQUEUE_FULL

The following macros are used throughout the FreeRTOS source code.

Macro	Value
pdTRUE	1
pdFALSE	0
pdPASS	1
pdFAIL	0

Memory Management

The RAM required to hold objects such as tasks, queues, semaphores, and event groups can be allocated statically at compile-time or dynamically at run-time. While dynamic allocation reduces design and planning effort, simplifies the API, and minimises the RAM footprint, static allocation is more deterministic, removes the need to handle memory allocation failures, and removes the risk of heap fragmentation – where the heap has enough free memory but not in one usable contiguous block.

Memory Allocation

The API functions which create kernel objects using statically allocated memory are available only when **configSUPPORT_STATIC_ALLOCATION** is set to 1 in FreeRTOSConfig.h.

Kernel objects using dynamically allocated memory are available and can be created only when **configSUPPORT_DYNAMIC_ALLOCATION** is set to 1 or left undefined in FreeRTOSConfig.h.

It is valid to have both constants set to 1 simulatenously.

Options for Dynamic Memory Allocation

Earlier versions of FreeRTOS used a memory pools allocations scheme.

Here, different sized memory blocks are pre-allocated at compile-time and returned by the memory allocation functions. So, memory allocation was a part of the core codebase. This is common in many real-time systems, but was removed from FreeRTOS because its inefficient use of RAM in really small embedded systems led to many support requests.

Now, FreeRTOS treats memory allocation as a part of the portable layer. Different embedded systems have different dynamic memory allocation and timing requirements, so a single dynamic memory allocation algorithm will never be able to cater to the variety of applications available all over. In addition, remove dynamic memory allocation from the core codebase enables application writers to provide their own specific implementations when appropriate.

The following functions are public functions that can be called from the application code, and are used by FreeRTOS instead of malloc() and free(); these functions have the same prototype as the aforementioned standard C library functions.

Function Name	Purpose
pvPortMalloc()	When FreeRTOS requires RAM.
vPortFree()	Frees previously allocated RAM

FreeRTOS has multiple implementations of these two functions. FreeRTOS applications can very well use these implementations or even go ahead and provide their own.

Some of the implementations are discussed below.

Heap_1

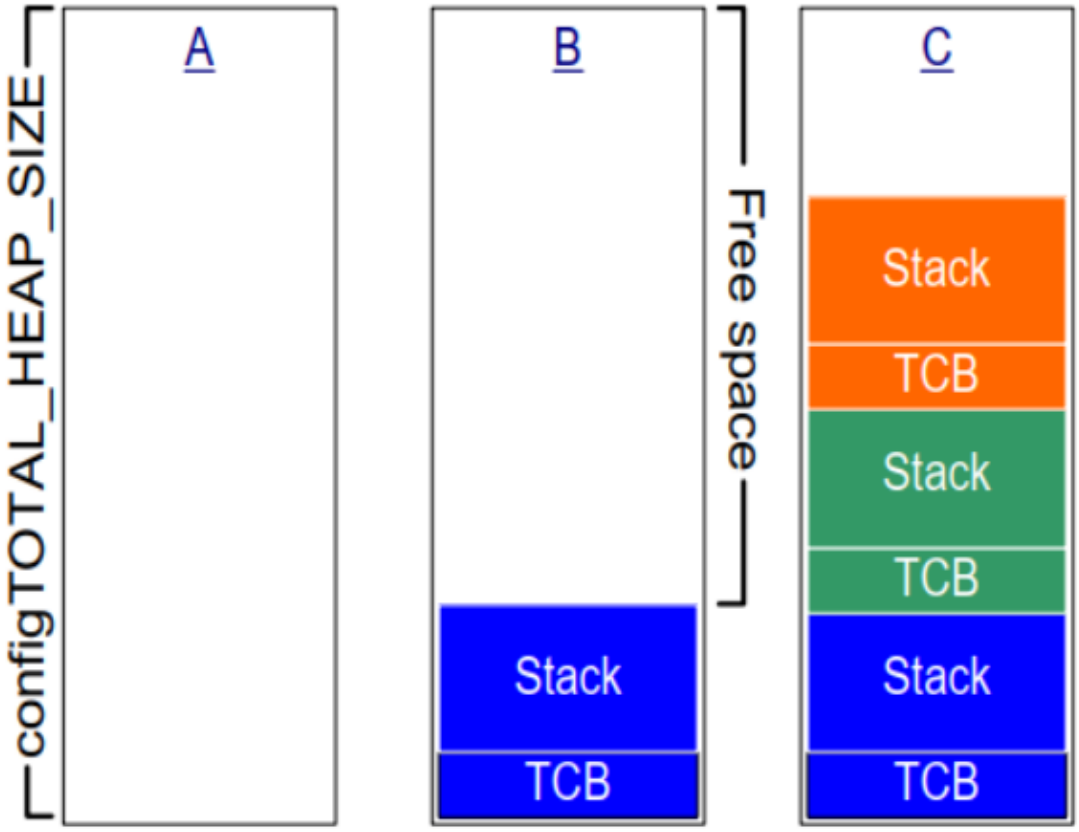
- Common for small, dedicated embedded systems to only create tasks and other kernel objects before starting the FreeRTOS scheduler.
- Memory only gets dynamically allocated before the application starts to perform any real-time functionality.
- Memory remains allocated for the application's lifetime.
- So, issues such as determinism and fragmentation are avoided, and attributes such as code size and simplicity are prioritised.

Heap_1.c implements a very basic version of pvPortMalloc(), and doesn't implement vPortFree(), meaning tasks or other kernel objects are never deleted. It is always deterministic and can't fragment memory.

A simple **uint8_t** array is subdivided into smaller blocks each time the pvPortMalloc() function is called. This array is called the FreeRTOS heap. The macro **configTOTAL_HEAP_SIZE** sets the size of the array in bytes.

This heap is NOT implemented as a statically allocated array, because when it is done that way, the heap becomes a part of the FreeRTOS data and that makes it appear to consume a lot of RAM.

Each dynamically allocated task results in two calls to pvPortMalloc(). The first one allocates a task control block (TCB), and the second one allocates the task's stack.



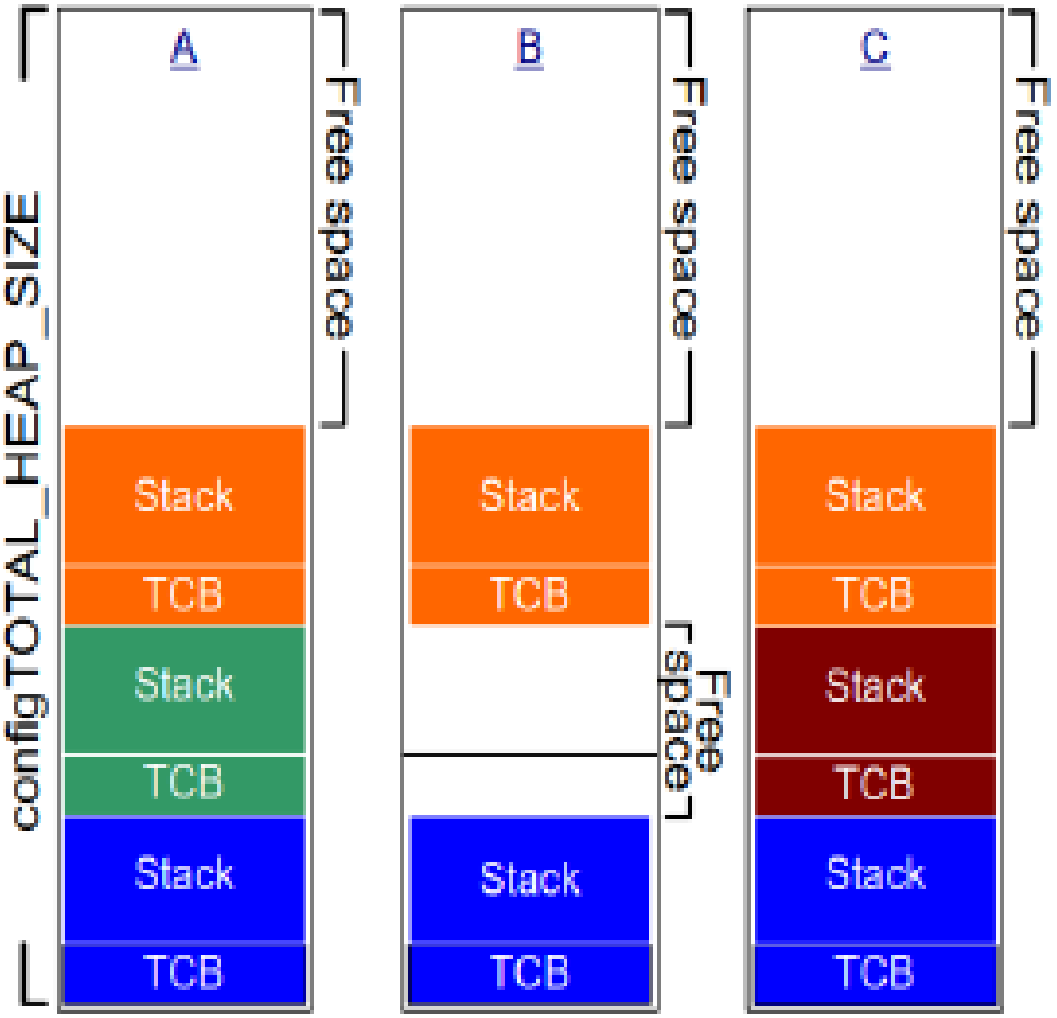
Here,

- **A** shows the array before creating any tasks.
- **B** shows the array after creating one task.
- **C** shows the array after creating three tasks.

Heap_2

Heap_2.c also works by subdividing an array that is dimensioned by the **configTOTAL_HEAP_SIZE** constant. It uses a best-fit algorithm to allocate memory, and unlike heap_1, it implements vPortFree().

The best-fit algorithm ensures that pvPortMalloc() uses the free block of memory that is closest in size to the number of bytes requested. Heap_2 doesn't combine adjacent free blocks into a single larger block, so it is susceptible to fragmentation. However, fragmentation is not an issue if the allocated and subsequently freed blocks are always the same size.



Here,

- **A** shows the array after allocating 3 tasks. A large free block remains at the top of the array.
- **B** shows the array after deleting one of the tasks. The large free block at the top of the array remains, but there are also now two smaller free blocks that previously held the TCB and stack of the deleted task.
- **C** shows the situation after creating another task. The best-fit algorithm reuses the block of RAM that held the stack of the deleted task to hold the stack of the newly created task, since they both require the same memory.

Heap_2 is not deterministic, but is faster than most standard library implementations of malloc() and free().

Heap_3

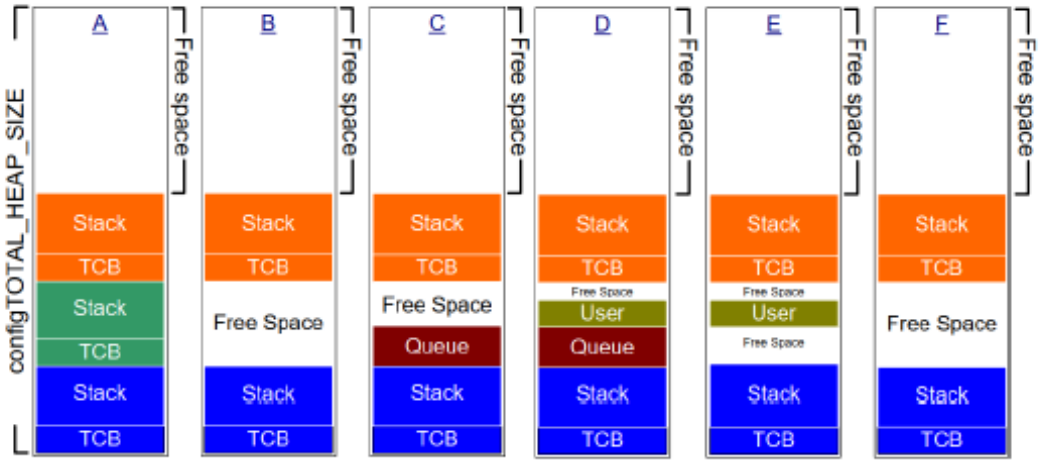
Heap_3.c uses the standard library malloc() and free() functions, so the linker configuration defines the heap size, and the configTOTAL_HEAP_SIZE constant is not used. Heap_3 makes malloc() and free() thread-safe by temporarily suspending the FreeRTOS scheduler for the duration of their execution.

Heap_4

Like heap_1 and heap_2, heap_4 works by subdividing an array into smaller blocks. As before, the array is statically allocated and dimensioned by configTOTAL_HEAP_SIZE, which makes FreeRTOS appear to use a lot of RAM as the heap becomes part of the FreeRTOS data.

Heap_4 uses a first-fit algorithm to allocate memory. Unlike heap_2, heap_4 combines or coalesces adjacent free blocks of memory into a single larger block, which minimises the risk of memory fragmentation.

The first-fit algorithm ensures pvPortMalloc() uses the first free block of memory that is large enough to hold the no. of bytes requested.



Here,

- **A** shows the array after creating 3 tasks. A large free block of memory remains at the top of the array.
- **B** shows the array after deleting one of the tasks. The large free block of memory at the top of the array remains. Unlike in the

heap_2 example, heap_4 merges the two memory blocks that previously held the TCB and stack of the deleted task, respectively, into a larger single free block.

- **C** shows the situation after creating a FreeRTOS queue. As heap_4 uses a first-fit algorithm, RAM is allocated from the first free RAM block that is large enough to hold the queue, which in this case is the RAM freed by deleting the task. The queue does not consume all the RAM in the free block, so the block is split into two, and the unused portion remains available to future calls to pvPortMalloc().
- **D** shows the situation after calling pvPortMalloc() directly from application code. The user allocated block is small enough to fit in the first free block, which is the block between the memory allocated to the queue and the memory allocated to the TCB following it.
- **E** shows the situation after deleting the queue, which automatically frees the memory allocated to the deleted queue. There is now free memory on either side of the user allocated block.
- **F** shows the situation after freeing the user allocated memory. The memory previously used by the user allocated block has been combined with the free memory on either side to create a larger, single free block.

Heap_5

Heap_5 uses the same allocation algorithm as heap_4.

Unlike heap_4, which is limited to allocating memory from a single array, heap_5 can combine memory from multiple separated memory spaces into a single heap.

Heap_5 is useful when the RAM provided by the system on which FreeRTOS is running does not appear as a single contiguous (without space) block in the system's memory map.

Static Memory Allocation

There are quite a number of disadvantages that come with dynamic memory allocation. A few notable ones are:

- They are not always available on small embedded systems.
- Their implementation can be large, taking up valuable code space.
- They are rarely thread-safe.
- They are not deterministic; the amount of time taken to execute the functions will differ from call to call.
- They can suffer from fragmentation.
- They can complicate the linker configuration.
- They can be the source of difficult to debug errors if the heap space is allowed to grow into memory used by other variables.

In static memory allocation, all the required memory is known at compile time, and is deterministic. A few complications exist here as well. The main complication is the addition of a few additional user functions to manage some kernel memory, and the second complication is the need to ensure all static memory is declared in a suitable scope.

When configSUPPORT_STATIC_ALLOCATION is set to 1, the kernel enables all the static versions of kernel functions. These are:

- xTaskCreateStatic
- xEventGroupCreateStatic
- xEventGroupGetStaticBuffer
- xQueueGenericCreateStatic
- xQueueGenericGetStaticBuffers
- xQueueCreateMutexStatic
    - if configUSE_MUTEXES is 1
- xQueueCreateCountingSemaphoreStatic
    - if configUSE_COUNTING_SEMAPHORES is 1
- xStreamBufferGenericCreateStatic
- xStreamBufferGetStaticBuffers
- xTimerCreateStatic
    - if configUSE_TIMERS is 1
- xTimerGetStaticBuffer
    - if configUSE_TIMERS is 1

The idle task and the timer task will use static memory supplied by user functions. These user functions are:

1) vApplicationGetTimerTaskMemory

Allows the application to create and return a memory buffer for the timer task TCB and the timer task stack. The function will also return the size of the time task stack.

```
void vApplicationGetTimerTaskMemory (StaticTask_t
                                     **ppxTimerTaskTCBBuffer,
                                     StackType_t
                                     **ppxTimerTaskStackBuffer,
                                     uint32_t
                                     *pulTimerTaskStackSize ) {

    static StaticTask_t xTimerTaskTCB;
    static StackType_t
    uxTimerTaskStack[configMINIMAL_STACK_SIZE];
    ppxTimerTaskTCBBuffer = &xTimerTaskTCB;
    ppxTimerTaskStackBuffer = uxTimerTaskStack;
    pulTimerTaskStackSize = sizeof(uxTimerTaskStack) /
    sizeof(*uxTimerTaskStack);
}
```

2) vApplicationGetIdleTaskMemory

The idle task is run when a core runs out of scheduled work. The idle task performs some housekeeping and can also trigger the user’s vTaskIdleHook() if it is enabled. In a symmetric multiprocessing system (SMP), there are also non-housekeeping idle tasks for each of the remaining cores, but these are statically allocated internally.

The vApplicationGetIdleTaskMemory function is called to allow the application to create the needed buffers for the "main" idle task.

```
void vApplicationGetIdleTaskMemory (StaticTask_t
                                    **ppxIdleTaskTCBBuffer,
                                    StackType_t
                                    **ppxIdleTaskStackBuffer
                                    , uint32_t
                                    *pulIdleTaskStackSize) {
    static StaticTask_t xIdleTaskTCB;
    static StackType_t
    uxIdleTaskStack[configMINIMAL_STACK_SIZE];
    ppxIdleTaskTCBBuffer = &xIdleTaskTCB;
    ppxIdleTaskStackBuffer = uxIdleTaskStack;
    pulIdleTaskStackSize = configMINIMAL_STACK_SIZE;
}
```

Task Management

A task is a small program that has an entry point, will normally run forever in an infinite loop, and doesn't exit. They are implemented as C functions.

```
void vATaskFunction (void * pvParameters) {
    long lStackVariable = 0;
    static long lStaticVariable = 0;
    for (;;) {}
    vTaskDelete(NULL);
}
```

A FreeRTOS task must not be allowed to return from the function that implements it in any way. It must not contain a *return* statement, and must not be allowed to execute past the end of its implementing function. If a task is no longer required, it should be explicity deleted.

A single task function definition can be used to create any number of tasks where each created task is a separate execution instance. Each instance has its own stack and thus its own copy of any automatic (stack) variables defined within the task itself.

A task is in the *Running* state when the processor is executing that task's code. It is in the *Not Running* state when it is paused and its state has been saved so that it may resume execution the next time the scheduler decides it should enter the *Running* state. When a task resumes execution, it does so from the instruction it was about to execute before it left the *Running* state.

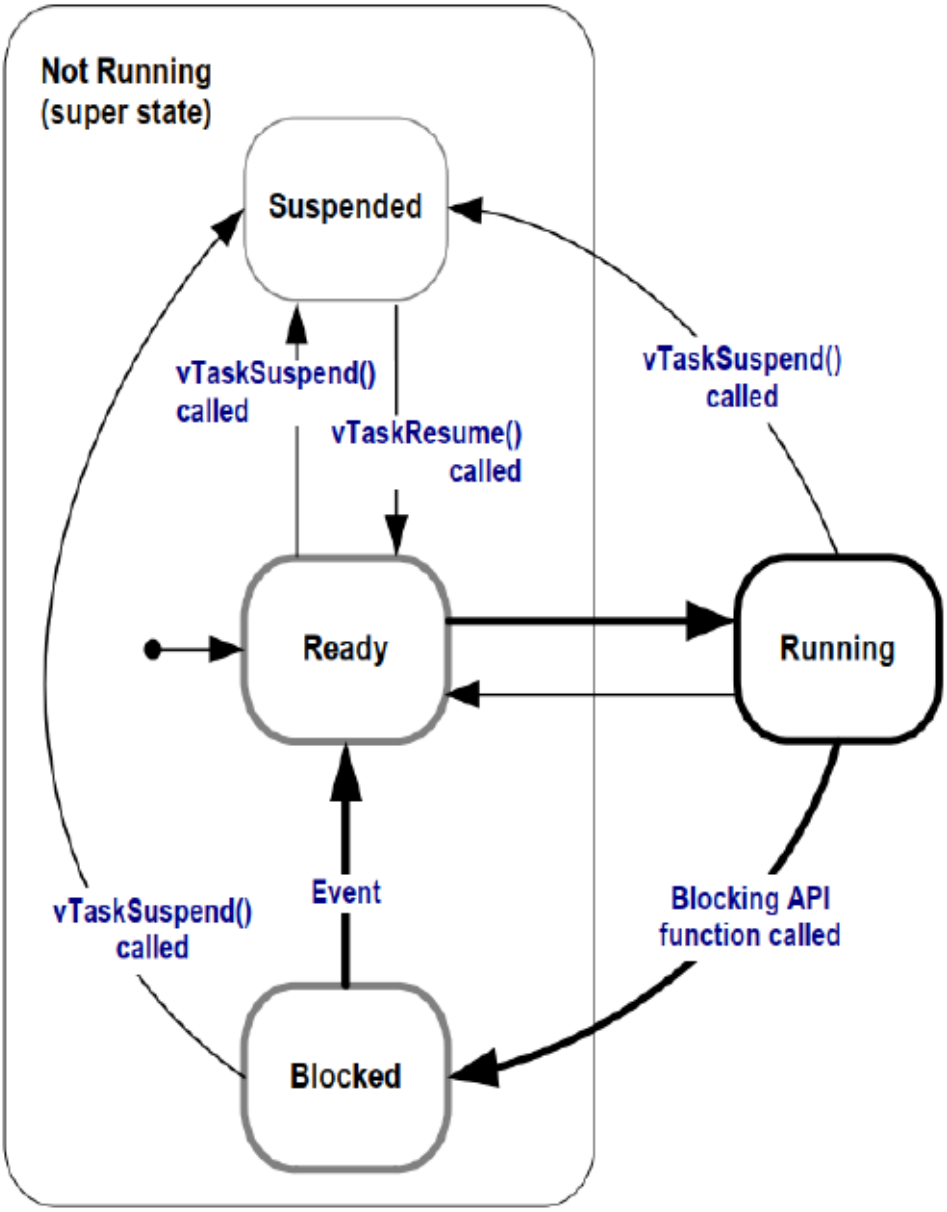
The transitioning of a task from the *Running* state to the *Not Running* state is called *swap out* or *switch out*. The transitioning of a task from the *Not Running* state to the *Running* state is called *swap in* or *switch in*.

Expanding the *Not Running* State

The FreeRTOS scheduler always ensures that the highest priority task that can run is the task selected to enter the *Running* state. Tasks of equal priority are transitioned into and out of the *Running* state in turn.

In a real-world scenario, the *Running* and *Not Running* states are not the only ones that exist. Some tasks have to wait for others to complete. Some tasks may be blocked or suspended depending on the circumstances. So, it is necessary to expand upon these.

To make tasks useful, they must be event-driven, i.e. a task has work to perform only after an event triggers it; it can't enter the *Running* state before that time. The scheduler always selects the highest priority task that can run. If a high-priority task can't be selected because it is waiting for an event, the scheduler must be able to select a lower-priority task that can run. Therefore, writing event-driven tasks means task can be created at different priorities without the highest priority tasks starving all the lower priority tasks of processing time.



Blocked State

A task waiting for an event is said to be in the *Blocked* state. Tasks can enter the *Blocked* state to wait for 2 different types of events:

1) Temporal Events

They occur either when a delay period expires or an absolute time is reached. For example, a task may enter the *Blocked* state to wait for 10 milliseconds to pass.

2) Synchronisation Events

These originate from another task or interrupt. For example, a task may enter the *Blocked* state to wait for data to arrive on a queue. Synchronisation events cover a broad range of event types.

Suspended State

Tasks in the *Suspended* state are not available to the scheduler. The only way to enter the *Suspended* state is through a call to the vTaskSuspend() API function, and the only way out is through a call to the vTaskResume() or xTaskResumeFromISR() functions. Most applications don't use the *Suspended* state.

Ready State

Tasks that are in the *Not Running* state, and are not *Blocked* or *Suspended* are said to be in the *Ready* state. They can run, and are therefore *ready* to run, but are not currently in the *Running* state.

The Idle Task

There must always be at least one task that can enter the *Running* state. To ensure that this is the case, the scheduler automatically creates an Idle Task when vTaskStartScheduler() is called. The idle task does very little more than sit in a loop, so it is always able to run. The idle task has the lowest possible priority (priority zero), to ensure it never prevents a higher priority application task from entering the Running state. However, there is nothing to prevent application designers from creating tasks at, and therefore that share, the idle task priority, if desired.

The Idle Task Hook

It is possible to add application specific functionality directly into the idle task through the use of an idle hook (or idle callback) function, which is a function that is called automatically by the idle task once per iteration of the idle task loop.

Common uses for the Idle Task Hook include:

- Executing low priority, background, or continuous processing functionality without the RAM overhead of creating application tasks for the purpose.
- Measuring the amount of spare processing capacity. The idle task runs only when all higher priority application tasks have no work to perform; so measuring the amount of processing time allocated to the idle task provides a clear indication of spare processing time.
- Placing the processor into a low power mode, providing an easy and automatic method of saving power whenever there is no application processing to be performed.

Idle Task Hook functions must adhere to the following rules:

- An Idle Task Hook function must never attempt to block or suspend itself. Blocking the idle task in any way could cause a scenario where no tasks are available to enter the *Running* state.
- If an application task uses the vTaskDelete() API function to delete itself, then the Idle Task Hook must return to its caller within a reasonable time period. This is because the Idle Task is responsible for cleaning up kernel resources allocated to tasks that delete themselves. If the idle task remains permanently in the Idle Hook function, then this clean-up cannot occur.

Changing the Priority of a Task

The vTaskPrioritySet() API function changes the priority of a task after the scheduler has been started. The vTaskPrioritySet() API function is only available when INCLUDE_vTaskPrioritySet is set to 1 in FreeRTOSConfig.h.

Deleting a Task

The vTaskDelete() API function deletes a task. The vTaskDelete() API function is only available when INCLUDE_vTaskDelete is set to 1 in FreeRTOSConfig.h.

Scheduling Algorithms

Scheduling algorithm is the software routine that decides which *Ready* state task to transition into the *Running* state.

The scheduling algorithm can be changed using the configUSE_PREEMPTION and configUSE_TIME_SLICING configuration constants, both of which are defined in FreeRTOSConfig.h. A third configuration constant, namely configUSE_TICKLESS_IDLE, also affects the scheduling algorithm, as its use can result in the tick interrupt being turned off completely for extended periods. configUSE_TICKLESS_IDLE is an advanced option provided specifically for use in applications that must minimise their power consumption.

Scheduling Algorithm	Prioritised	configUSE_PREEMPTION	configUSE_TIME_SLICING
Preemptive with Time Slicing	Yes	1	1
Preemptive without Time Slicing	Yes	1	0
Co-operative	No	0	Any

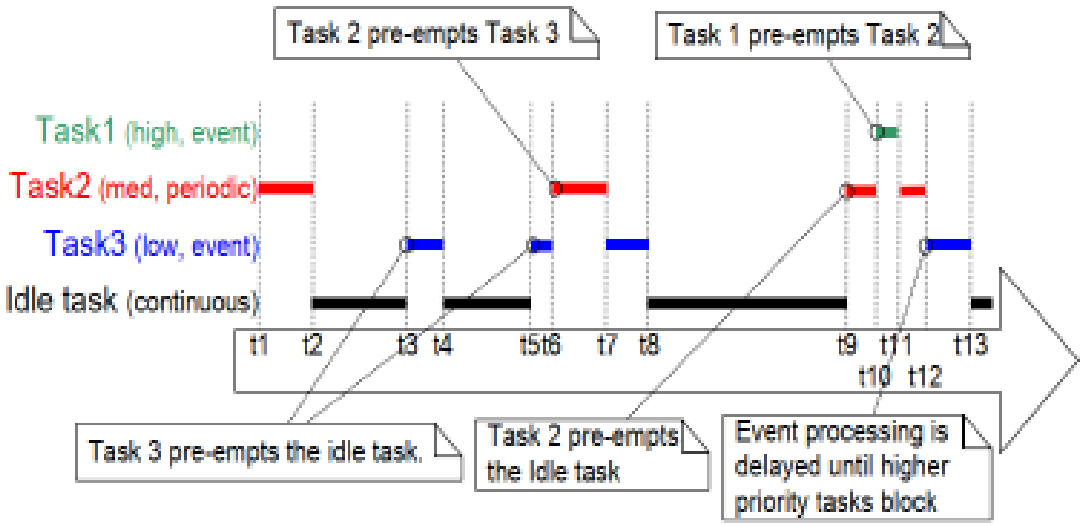
Prioritised Preemptive Scheduling with Time Slicing

This is the scheduling algorithm used by most small RTOS applications. It is a fixed priority, preemptive algorithm. It doesn’t change the priority assigned to the tasks being scheduled, and doesn’t prevent the tasks themselves from changing their own priority or that of other tasks.

It is preemptive, and so will immediately preempt the *Running* state if a task that has a priority higher than the *Running* state task enters the *Ready* state, so

that this new task can enter the *Running* state. Task preemption can occur at any time, not just in the RTOS tick interrupt.

This algorithm also makes use of time slicing. Time slicing is used to share processing time between tasks of equal priority, even when the tasks don't explicitly yield or enter the *Blocked* state. Scheduling algorithms described as using time slicing select a new task to enter the *Running* state at the end of each time slice if there are there are other *Ready* state tasks that have the same priority as the *Running* state task. A time slice is equal to the time between two RTOS tick interrupts.



Prioritised Preemptive Scheduling without Time Slicing

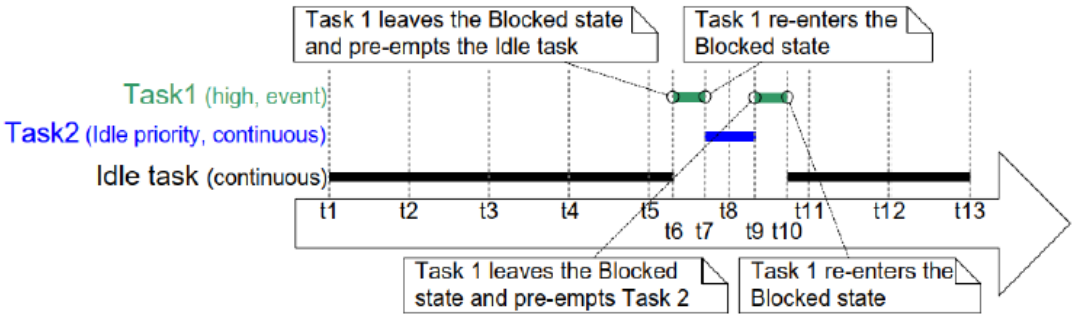
Prioritised Preemptive Scheduling without time slicing maintains the same task selection and pre-emption algorithms as described above, but doesn't use time slicing to share processing time between tasks of equal priority.

Now, a scheduler does NOT select a new task to enter the *Running* state during each RTOS tick interrupt. The scheduler selects a new task to enter the *Running* state when either

- a higher priority task enters the *Ready* state.
- the task in the *Running* state enters the *Blocked* or *Suspended* state.

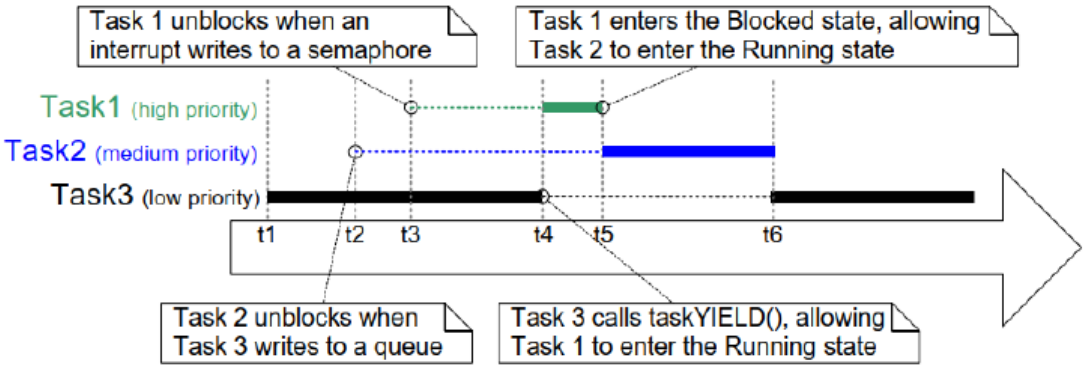
There are fewer context switches when time slicing is not used than when time slicing is used. Therefore, turning time slicing off results in a reduction in the

scheduler's processing overhead. However, turning time slicing off can also result in tasks of equal priority receiving greatly different amounts of processing time. For this reason, running the scheduler without time slicing is considered an advanced technique that should only be used by experienced users.



Cooperative Scheduling

FreeRTOS can also use cooperative scheduling. When using the cooperative scheduler, it is assumed that application-provided interrupt service routines don't explicitly request context switches. A context switch only occurs when the *Running* state task enters the *Blocked* state, or the *Running* state task explicitly yields, i.e. manually requests a re-schedule, by calling taskYIELD(). Tasks are never preempted, so time slicing can't be used.



Using the cooperative scheduler normally makes it easier to avoid problems caused by simultaneous access than when using preemptive schedulers.

While using a preemptive scheduler, the *Running* state task can be pre-empted at any time, including when a resource it is sharing with another task is in an inconsistent state. But when a cooperative scheduler is used, you control when a

switch to another task occurs. So, here, it can be ensured that a switch to another task doesn’t occur while a resource is in an inconsistent state.

There are drawbacks to the cooperative scheduler as well. Using the cooperative scheduler makes systems less responsive than when using the preemptive scheduler.

When using the preemptive scheduler, the scheduler starts running a task immediately when the task becomes the highest priority *Ready* state task. This is often essential in real-time systems which must respond to high priority events within a defined time period.

When using the cooperative scheduler, since there is no preemption, a switch to a task that has become the highest priority *Ready* state task is not performed until the *Running* state task enters the *Blocked* state or calls taskYIELD().

Queue Management

Queues are First In First Out (FIFO) buffers, where data is written to the end (tail) of the queue and removed from the front (head) of the queue. They can hold a finite number of fixed size data items. The maximum no. of items a queue can hold is called its *length*. Both the length and size of each data item are set when the queue is created. Queues provide the following communication mechanisms:

- task-to-task
- task-to-interrupt
- interrupt-to-task

There are 2 ways in which queue behaviour can be implemented:

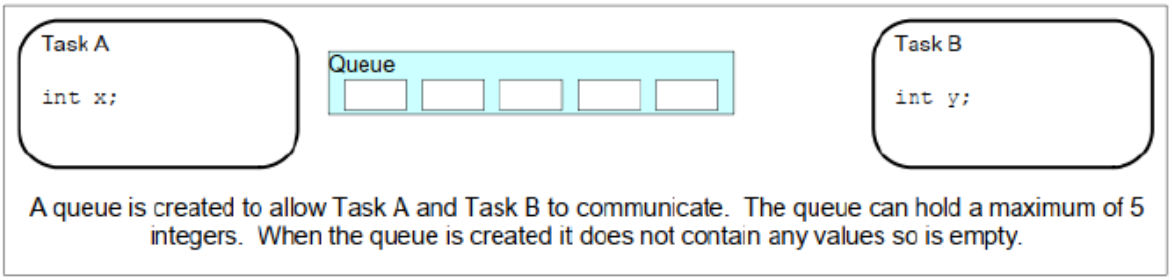
1) Queue by Copy: Data sent to the queue is copied byte for byte into the queue.
2) Queue by Reference: The queue only holds pointers to the data sent to it.

FreeRTOS uses the queue by copy method, because it is both simpler and more powerful to use than queueing by reference.

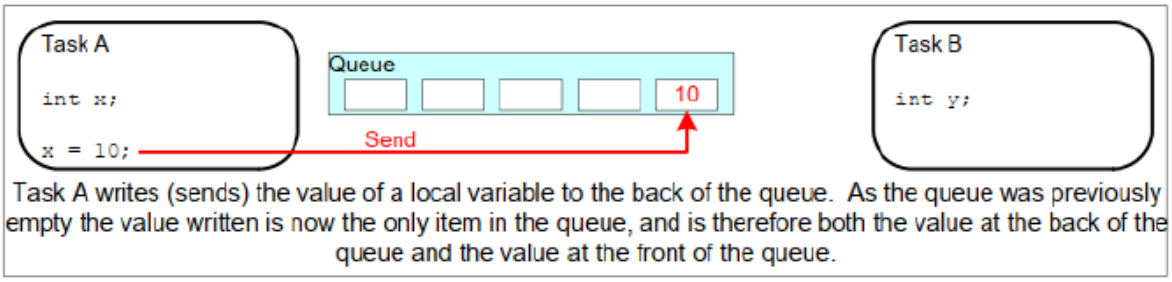
Queueing by copy doesn't prevent the queue from also being used to queue by reference. For example, when the size of the data being queued becomes very large, it becomes impractical to copy the data into the queue. So, a pointer to the data can be copied into the queue instead.

Data can be sent to a queue without first allocating a buffer to hold the data - then the data is copied into the allocated buffer and a reference is queued to the buffer. The sending task can immediately reuse the variable or buffer that was sent to the queue; since the sending task and receiving task are completely decoupled, an application designer doesn't need to concern themselves with which task *owns* the data, or which task is responsible for releasing the data.

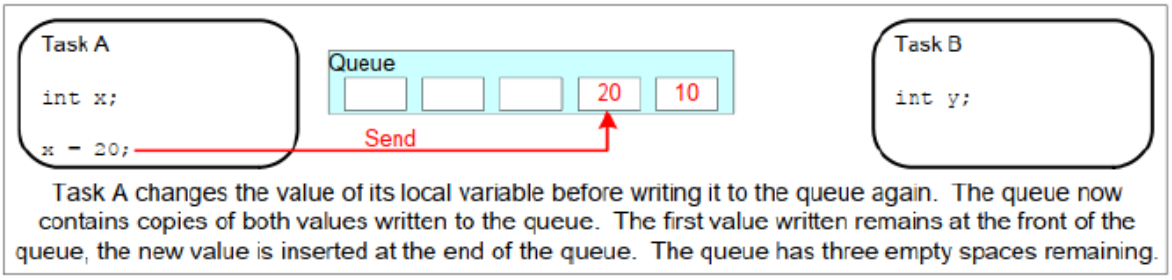
Memory protected systems restrict access to RAM, in which case queueing by reference can only be accomplished if the sending and receiving tasks can both access the referenced data. Queueing by copy allows data to pass across memory protection boundaries.



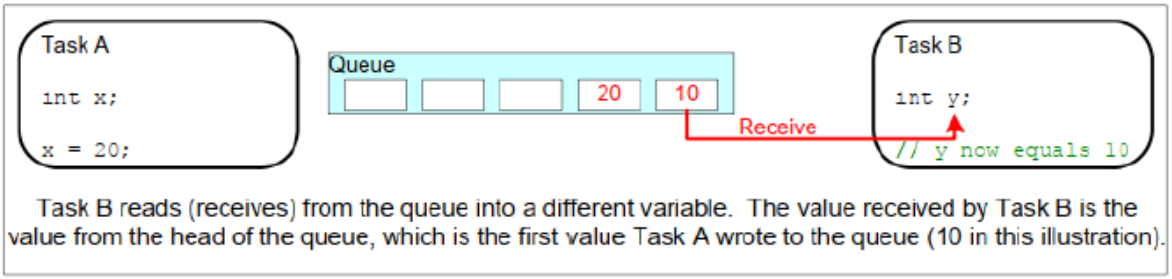
A queue is created to allow Task A and Task B to communicate. The queue can hold a maximum of 5 integers. When the queue is created it does not contain any values so is empty.



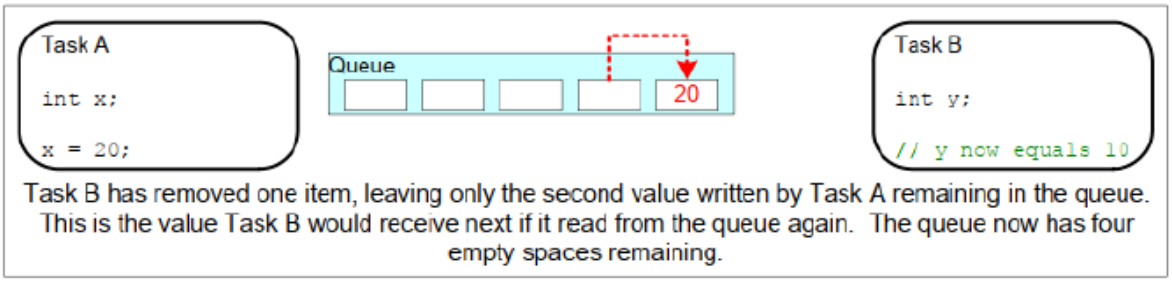
Task A writes (sends) the value of a local variable to the back of the queue. As the queue was previously empty the value written is now the only item in the queue, and is therefore both the value at the back of the queue and the value at the front of the queue.



Task A changes the value of its local variable before writing it to the queue again. The queue now contains copies of both values written to the queue. The first value written remains at the front of the queue, the new value is inserted at the end of the queue. The queue has three empty spaces remaining.



Task B reads (receives) from the queue into a different variable. The value received by Task B is the value from the head of the queue, which is the first value Task A wrote to the queue (10 in this illustration).



Task B has removed one item, leaving only the second value written by Task A remaining in the queue. This is the value Task B would receive next if it read from the queue again. The queue now has four empty spaces remaining.

Queues are objects in their own right; any number of tasks can write to the same queue, and any number of tasks can read from the same queue. In practice, it's very common for a queue to have multiple writers, but much less common for it to have multiple readers.

Blocking on Queue Reads

When a task attempts to read from a queue, it can optionally specify a *block* time. This is the time that the task is kept in the *Blocked* state to wait for the data to become available from the queue. A task in the *Blocked* state, waiting for data to become available from the queue, is automatically moved to the *Ready* state when another task or interrupt places data into the queue. The task is also automatically moved from the *Blocked* to the *Ready* state if the specified block time expires before the data becomes available.

Queues can have multiple readers, so it is possible for a single queue to have more than one task blocked on it, waiting for some data. When this is the case, only one task is unblocked when data becomes available. The task that is unblocked is always the highest priority task that is waiting for the data. If two or more blocked tasks have equal priority, then the task that is unblocked is the one that has been waiting the longest.

Blocking on Queue Reads

Just as it can when reading from a queue, a task can optionally specify a block time when writing to a queue. In this case, the block time is the maximum time the task will be held in the *Blocked* state to wait for space to become available on the queue, should the queue already be full.

Queues can have multiple writers, so it’s possible for a full queue to have more than one task blocked on it, waiting to complete a send operation. When this is the case, only one task is unblocked when space on the queue becomes available. The task that is unblocked is always the highest priority task that is waiting for space. If two or more blocked tasks have equal priority, then the task that is unblocked is the one that has been waiting the longest.

Blocking on Multiple Queues

Queues can be grouped into sets, allowing a task to enter the *Blocked* state to wait for data to become available on any of the queues in the set. Queue set designs are less neat, less efficient than single queues.

Software Timer Management

Software timers are used to schedule the execution of a function at a set time in the future, or periodically with a fixed frequency. The function executed by the software timer is called the software timer's callback function.

Software timers are implemented by, and under the control of, the FreeRTOS kernel. They don't require hardware support, and aren't related to hardware timers or hardware counters. They don't use any processing time unless the callback function is actually executing.

Software timer functionality is optional. To include it:

1) Build the FreeRTOS source file FreeRTOS/Source/timers.c as part of the project.

2) Define the following constants present in the FreeRTOSConfig.h header file:

- configUSE_TIMERS: Set to 1
- configTIMER_TASK_PRIORITY: Sets priority of the timer service task between 0 and configMAX_PRIORITIES-1.
- configTIMER_QUEUE_LENGTH: Sets the maximum no. of unprocessed commands that the timer command queue can hold at any one time.
- configTIMER_TASK_STACK_DEPTH: Sets the size of the stack allocated to the timer service task.

Attributes of a Software Timer

1) Period

It is the time between the software timer being started, and the software timer's callback function executing.

2) One-Shot Timer

Once started, a one-shot timer will execute its callback function only once. It can be restarted manually, but won’t restart itself.

3) Auto-Reload Timer

Once started, an auto-reload timer will restart itself each time it expires, resulting in periodic execution of its callback function.

Software Timer States

1) Dormant

A dormant software timer exists, and can be referenced by its handle, but is not running. So, its callback functions will not execute.

2) Running

A running software timer will execute its callback function after a time equal to its period has elapsed since the software time entered the *Running* state, or since the software timer was reset.

The RTOS Daemon Task

The daemon timer service task is a standard FreeRTOS task that’s created automatically when the scheduler is started. Its priority is set by the configTIMER_TASK_PRIORITY and its stack size or stack depth by the configTIMER_TASK_STACK_DEPTH configuration constant, both of which are defined within FreeRTOSConfig.h.

All software timer callback functions execute in the context of the same RTOS daemon task.

Interrupt Management

Embedded real-time systems have to take actions in response to events that originate from the environment.

Non-trivial systems will have to service events that originate from multiple sources, all of which will have different processing overhead and response time requirements.

In each case, a judgment has to be made as to the best event processing implementation strategy. Interrupts are normally used in order to detect such events. FreeRTOS doesn't impose any specific event processing strategy on the application designer, but does provide features that allow the chosen strategy to be implemented in a simple and maintainable way.

It's important to draw a distinction between the priority of a task and the priority of an interrupt:

- A task is a software feature that is unrelated to the hardware on which FreeRTOS is running. Its priority is assigned in software by the application writer, and the schedule decides which task will be placed in the *Running* state.
- Although written in software, an interrupt service routine (ISR) is a hardware feature, because the hardware controls which interrupt service routine will run and when it will run. Tasks will only run when there are no ISRs running, so the lowest priority interrupt will interrupt the highest priority task, and there is no way for a task to preempt an ISR.

All architectures on which FreeRTOS will run are capable of processing interrupts, but details relating to interrupt entry, and interrupt priority assignment, vary between architectures.

The Interrupt Safe API

Often, it is necessary to use the functionality provided by a FreeRTOS API function from an ISR, but many FreeRTOS API functions perform actions that are not valid inside an ISR.

The most notable of these is placing the task that called the API function into the *Blocked* state— if an API function is called from an ISR, then it is not being called from a task, so there is no calling task that can be placed into the *Blocked* state. FreeRTOS solves this problem by providing two versions of some API functions; one version for use from tasks, and one version for use from ISRs. Functions intended for use from ISRs have *FromISR* appended to their name.

Benefits

Having a separate API for use in interrupts allows task code to be more efficient, ISR code to be more efficient, and interrupt entry to be simpler. To see why, consider the alternative solution, which would have been to provide a single version of each API function that could be called from both a task and an ISR. If the same version of an API function could be called from both a task and an ISR then:

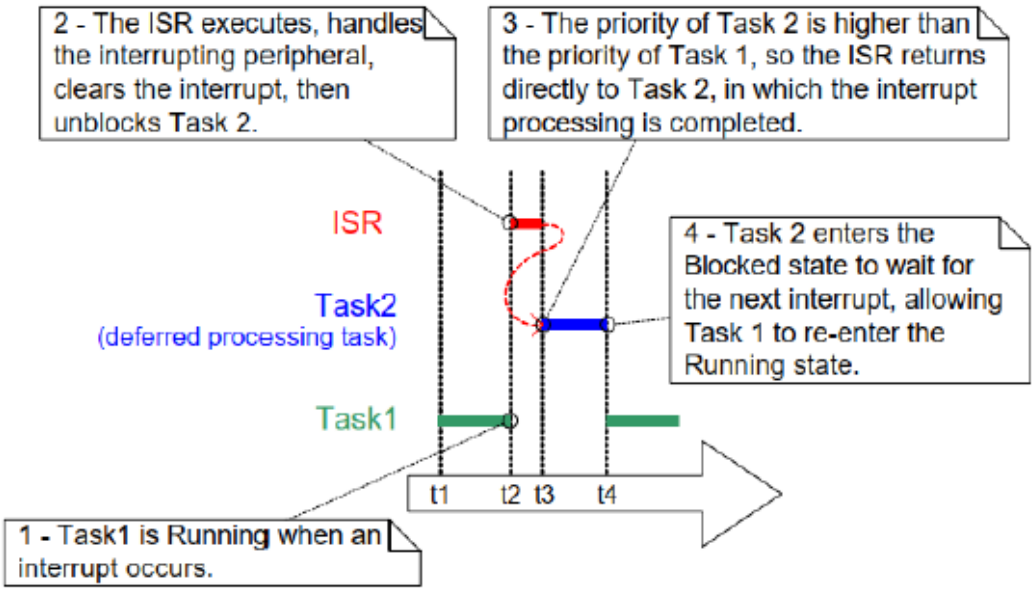
- The API functions would need additional logic to determine if they had been called from a task or an ISR. The additional logic would introduce new paths through the function, making the functions longer, more complex, and harder to test.
- Some API function parameters would be obsolete when the function was called from a task, while others would be obsolete when the function was called from an ISR.
- Each FreeRTOS port would need to provide a mechanism for determining the execution context (task or ISR).
- Architectures on which it is not easy to determine the execution context (task or ISR) would require additional, wasteful, more complex to use, and non-standard interrupt entry code that allowed the execution context to be provided by software.

Disadvantages

Having two versions of some API functions allows both tasks and ISRs to be more efficient, but introduces a new problem; sometimes it is necessary to call a function that is not part of the FreeRTOS API, but makes use of the FreeRTOS API, from both a task and an ISR.

This is normally only a problem when integrating third party code, as that is the only time when the software's design is out of the control of the application writer. If this does become an issue, then the problem can be overcome using one of the following techniques:

- Defer interrupt processing to a task, so the API function is only ever called from the context of a task.
- If you are using a FreeRTOS port that supports interrupt nesting, then use the version of the API function that ends in "FromISR", as that version can be called from tasks and ISRs. (The reverse is not true, API functions that do not end in "FromISR" must not be called from an ISR.)
- Third party code normally includes an RTOS abstraction layer that can be implemented to test the context from which the function is being called (task or interrupt), and then call the API function that is appropriate for the context.



There is no absolute rule as to when it is best to perform all processing necessitated by an interrupt in the ISR, and when it is best to defer part of the processing to a task. Deferring processing to a task is most useful when:

- The processing necessitated by the interrupt is not trivial. For example, if the interrupt is just storing the result of an analog to digital conversion, then it is almost certain this is best performed inside the ISR, but if result of the conversion must also be passed through a software filter, then it may be best to execute the filter in a task.
- It is convenient for the interrupt processing to perform an action that cannot be performed inside an ISR, such as write to a console, or allocate memory.
- The interrupt processing is not deterministic—meaning it is not known in advance how long the processing will take.

Context switches don't occur automatically inside the interrupt safe version of an API function. There are several reasons why:

- **Avoiding unnecessary context switches:** An interrupt may execute more than once before it is necessary for a task to perform any processing.
- **Control over the execution sequence:** Interrupts can occur sporadically, and at unpredictable times. Expert FreeRTOS users may want to temporarily avoid an unpredictable switch to a different task at specific points in their application, although this can also be achieved using the FreeRTOS scheduler locking mechanism.
- **Portability:** It is the simplest mechanism that can be used across all FreeRTOS ports.
- **Efficiency:** Ports that target smaller processor architectures only allow a context switch to be requested at the very end of an ISR, and removing that restriction would require additional and more complex code. It also allows more than one call to a FreeRTOS API function within the same ISR without generating more than one request for a context switch within the same ISR.
- **Execution in the RTOS tick interrupt:** It is possible to add application code into the RTOS tick interrupt. The result of attempting a context switch inside the tick interrupt is dependent on the FreeRTOS port in use. At best, it will result in an unnecessary call to the scheduler.

Binary Semaphores

The interrupt safe version of the Binary Semaphore API can be used to unblock a task each time a particular interrupt occurs, effectively synchronising the task with the interrupt. This allows the majority of the interrupt event processing to be implemented within the synchronised task, with only a very fast and short portion remaining directly in the ISR.

The binary semaphore can be used to *defer* interrupt processing to a task.

'Taking a semaphore' and 'giving a semaphore' are concepts that have different meanings depending on their usage scenario. In this interrupt synchronization scenario, the binary semaphore can be considered conceptually as a queue with a length of one. The queue can contain a maximum of one item at any time, so is always either empty or full (hence, binary).

By calling xSemaphoreTake() – the function which *obtains* or *receives* the semaphore – the task to which interrupt processing is deferred effectively attempts to read from the queue with a block time, causing the task to enter the *Blocked* state if the queue is empty.

When the event occurs, the ISR uses the xSemaphoreGiveFromISR() function – using which binary and counting semaphores can be *given* – to place a token (the semaphore) into the queue, making the queue full.

This causes the task to exit the *Blocked* state and remove the token, leaving the queue empty once more. When the task has completed its processing, it once more attempts to read from the queue and, finding the queue empty, re-enters the *Blocked* state to wait for the next event.

Counting Semaphores

Just as binary semaphores can be thought of as queues that have a length of one, counting semaphores can be thought of as queues that have a length of more than one. Tasks are not interested in the data that is stored in the queue – just the number of items in the queue. configUSE_COUNTING_SEMAPHORES must be set to 1 in FreeRTOSConfig.h for counting semaphores to be available.

Each time a counting semaphore is *given*, another space in its queue is used. The number of items in the queue is the semaphore's *count* value.

Counting semaphores are typically used for 2 things:

1) Counting Events

In this scenario, an event handler will *give* a semaphore each time an event occurs, causing the semaphore's count value to be incremented on each *give*. A task will *take* a semaphore each time it processes an event, causing the semaphore's count value to be decremented on each *take*. The count value is the difference between the number of events that have occurred and the number that have been processed.

Counting semaphores that are used to count events are created with an initial count value of zero.

2) Resource Management

In this scenario, the count value indicates the number of resources available. To obtain control of a resource, a task must first obtain a semaphore, which decrements the semaphore's count value. When the count value reaches zero, there are no free resources. When a task finishes with the resource, it 'gives' the semaphore back, which increments the semaphore's count value.

Counting semaphores that are used to manage resources are created so that their initial count value equals the number of resources that are available.

Resource Management

In a multitasking system, there is potential for error if one task starts to access a resource, but doesn't complete its access before being transitioned out of the *Running* state. If the task leaves the resource in an inconsistent state, then access to the same resource by any other task or interrupt could result in data corruption, or other similar issues.

To ensure data consistency is maintained at all times, access to a resource that is shared between tasks, or is shared between tasks and interrupts, must be managed using a **mutual exclusion** technique. The goal is to ensure that, once a task starts to access a shared resource that is not re-entrant and not thread-safe, the same task has exclusive access to the resource until the resource has been returned to a consistent state.

FreeRTOS provides several features that can be used to implement mutual exclusion, but the best mutual exclusion method is to (whenever possible, as it is often not practical) design the application in such a way that resources are not shared, and each resource is accessed only from a single task.

Basic Critical Sections

Basic critical sections are regions of code that are surrounded by calls to the macros taskENTER_CRITICAL() and taskEXIT_CRITICAL(), respectively. Critical sections are also known as critical regions. Calling taskENTER_CRITICAL() and taskEXIT_CRITICAL() is the only legitimate way for a task to alter the interrupt enable state of the processor on which FreeRTOS is running.

Critical sections implemented in this way are a crude method of providing mutual exclusion. They work by disabling interrupts either completely or up to the interrupt priority set by configMAX_SYSCALL_INTERRUPT_PRIORITY, depending on the FreeRTOS port being used. Pre-emptive context switches can occur only from within an interrupt, so, as long as interrupts remain disabled, the task that called taskENTER_CRITICAL() is guaranteed to remain in the Running state until the critical section is exited.

Critical sections can also be created by suspending the scheduler. Suspending the scheduler is sometimes also known as 'locking' the scheduler.

Basic critical sections protect a region of code from access by other tasks and by interrupts, but a critical section implemented by suspending the scheduler only protects a region of code from access by other tasks, because interrupts remain enabled.

A critical section that is too long to be implemented by simply disabling interrupts can, instead, be implemented by suspending the scheduler. However, interrupt activity while the scheduler is suspended can make resuming (or 'un-suspending') the scheduler a relatively long operation, so consideration must be given to which is the best method to use in each case.

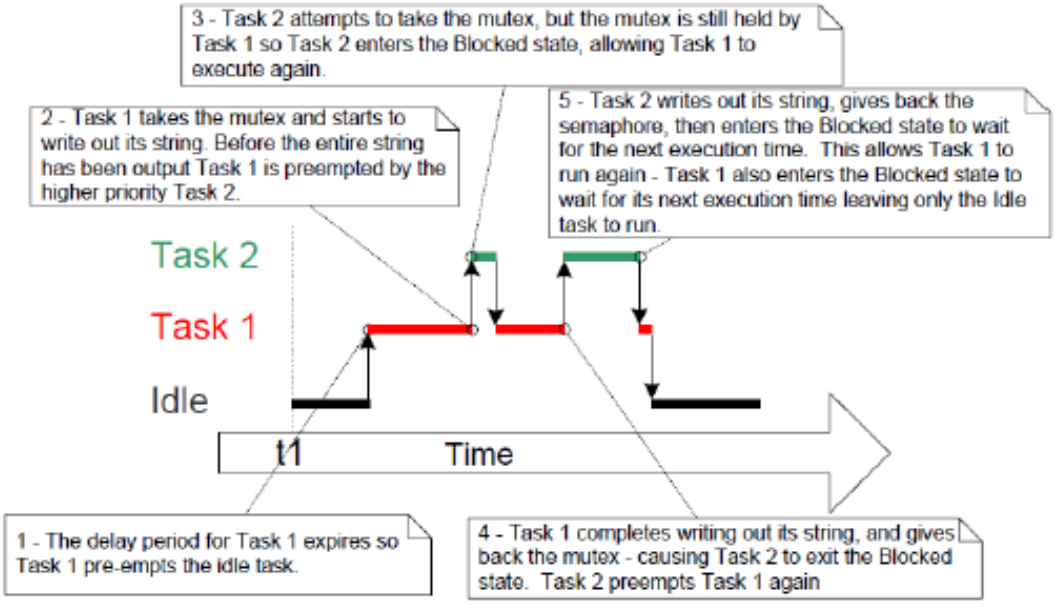
The scheduler is suspended by calling vTaskSuspendAll(). Suspending the scheduler prevents a context switch from occurring, but leaves interrupts enabled. If an interrupt requests a context switch while the scheduler is suspended, then the request is held pending, and is performed only when the scheduler is resumed (un-suspended).

Mutex

A Mutex is a special type of binary semaphore that is used to control access to a resource that is shared between two or more tasks. The word MUTEX originates from *MUTual EXclusion*. configUSE_MUTEXES must be set to 1 in FreeRTOSConfig.h for mutexes to be available.

When used in a mutual exclusion scenario, the mutex can be thought of as a token that is associated with the resource being shared. For a task to access the resource legitimately, it must first successfully 'take' the token (be the token holder). When the token holder has finished with the resource, it must 'give' the token back. Only when the token has been returned can another task successfully take the token, and then safely access the same shared resource. A task is not permitted to access the shared resource unless it holds the token.

A semaphore that is used for mutual exclusion must always be returned. A semaphore that is used for synchronization is normally discarded and not returned.



Priority Inversion

A higher priority task being delayed by a lower priority task is called *priority inversion*. This undesirable behavior would be exaggerated further if a medium priority task started to execute while the high priority task was waiting for the semaphore—the result would be a high priority task waiting for a low priority task—without the low priority task even being able to execute.

Priority Inheritance

FreeRTOS mutexes and binary semaphores are very similar—the difference being that mutexes include a basic *priority inheritance* mechanism, whereas binary semaphores do not. Priority inheritance is a scheme that minimizes the negative effects of priority inversion. It does not *fix* priority inversion, but merely lessens its impact by ensuring that the inversion is always time bounded. However, priority inheritance complicates system timing analysis, and it is not good practice to rely on it for correct system operation.

Priority inheritance works by temporarily raising the priority of the mutex holder to the priority of the highest priority task that is attempting to obtain the same mutex. The low priority task that holds the mutex 'inherits' the priority of the task waiting for the mutex. The priority of the mutex holder is reset automatically to its original value when it gives the mutex back.

Deadlock and Recursive Mutexes

Deadlock is another potential pitfall of using mutexes for mutual exclusion. Deadlock is sometimes also known by the more dramatic name *deadly embrace*. It occurs when 2 tasks can't proceed because they are both waiting for a resource that is held by the other.

It is also possible for a task to deadlock with itself. This happens if a task attempts to take the same mutex more than once, without first returning the mutex.

Considering the following scenario:

1) A task successfully obtains a mutex.
2) While holding the mutex, the task calls library function.
3) The implementation of the library function attempts to take the same mutex, and enters the *Blocked* state to wait for the mutex to become available.

At the end of this scenario the task is in the Blocked state to wait for the mutex to be returned, but the task is already the mutex holder. A deadlock has occurred because the task is in the *Blocked* state to wait for itself.

This type of deadlock can be avoided by using a *recursive mutex* in place of a standard mutex. A recursive mutex can be *taken* more than once by the same task, and will be returned only after one call to *give* the recursive mutex has been executed for every preceding call to *take* the recursive mutex.

Gatekeeper Tasks

Gatekeeper tasks provide a clean method of implementing mutual exclusion without the risk of priority inversion or deadlock. A gatekeeper task is a task that has sole ownership of a resource. Only the gatekeeper task is allowed to access the resource directly—any other task needing to access the resource can do so only indirectly by using the services of the gatekeeper. The gatekeeper task uses a FreeRTOS queue to serialize access to standard out. The internal implementation of the task does not have to consider mutual exclusion because it is the only task permitted to access standard out directly.

Other Features of FreeRTOS

Event Groups

Real-time embedded systems have to take actions in response to events. Previously, features of FreeRTOS that allow events to be communicated to tasks have been discussed. Examples of such features include semaphores and queues, both of which have the following properties:

- They allow a task to wait in the Blocked state for a single event to occur.
- They unblock a single task when the event occurs. The task that is unblocked is the highest priority task that was waiting for the event.

Event groups are another feature of FreeRTOS that allows events to be communicated to tasks. Unlike queues and semaphores,

- Event groups allow a task to wait in the *Blocked* state for a combination of one of more events to occur.
- Event groups unblock all the tasks that were waiting for the same event, or combination of events, when the event occurs.

These unique properties of event groups make them useful for synchronizing multiple tasks, broadcasting events to more than one task, allowing a task to wait in the *Blocked* state for any one of a set of events to occur, and allowing a task to wait in the *Blocked* state for multiple actions to complete.

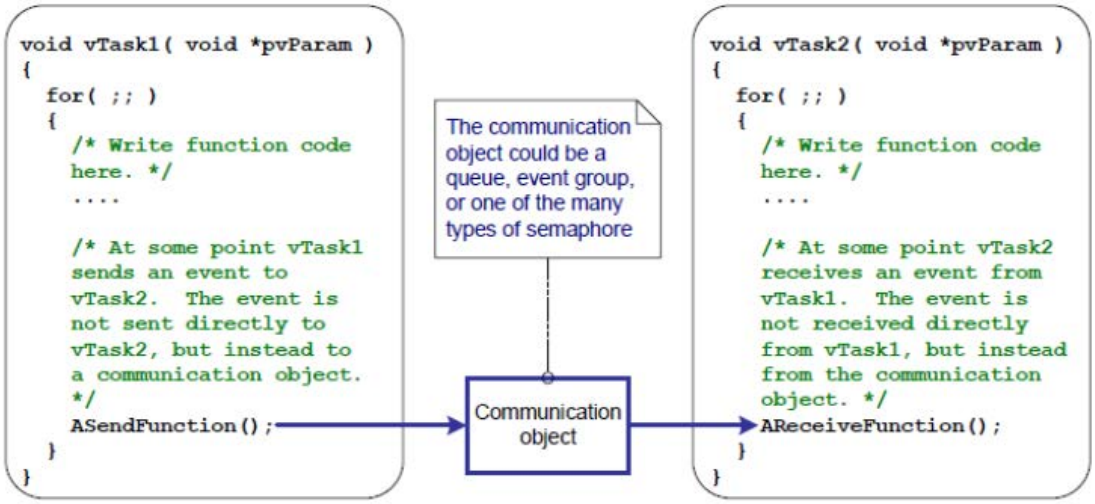
Event groups also provide the opportunity to reduce the RAM used by an application as, often, it is possible to replace many binary semaphores with a single event group. Event group functionality is optional. To include event group functionality, the FreeRTOS source file event_groups.c has to be built as part of the project.

Task Notifications

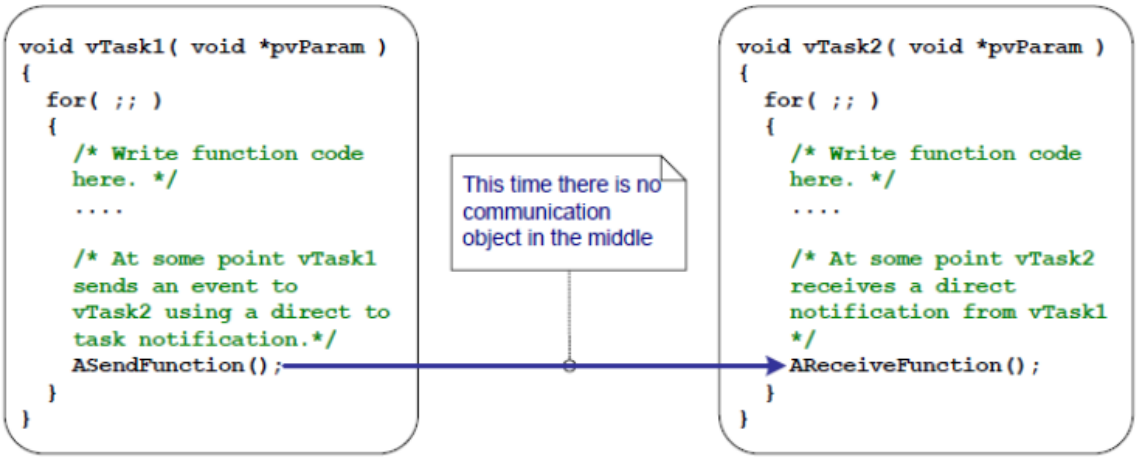
FreeRTOS applications are typically structured as a series of independent tasks that communicate with each other to collectively provide the system functionality. Task notifications are an efficient mechanism allowing one task to directly notify another task.

In order to communicate with each other, the creation of a communication object is required. Examples of these include queues, event groups, and various types of semaphores.

When a communication object is used, events and data are not sent directly to a receiving task, or a receiving ISR, but are instead sent to the communication object. Likewise, tasks and ISRs receive events and data from the communication object, rather than directly from the task or ISR that sent the event or data.



Task notifications exist to bypass these communication objects. They allow tasks to interact with other tasks, and to synchronise with ISRs, without the need for a separate communication object. By using a task notification, a task or ISR can send an event directly to the receiving task.



Task notification functionality is optional. To include task notification functionality set configUSE_TASK_NOTIFICATIONS to 1 in FreeRTOSConfig.h.

When configUSE_TASK_NOTIFICATIONS is set to 1, each task has at least one *Notification State*, which can be either *Pending* or *Not-Pending*, and a *Notification Value*, which is a 32-bit unsigned integer. When a task receives a notification, its notification state is set to pending. When a task reads its notification value, its notification state is set to not-pending. If the configTASK_NOTIFICATION_ARRAY_ENTRIES is set to a value > 1, then there are multiple notification states and values identified by index.

A task can wait in the *Blocked* state, with an optional time out, for its notification state to become pending.

Using a task notification to send an event or data to a task is significantly faster and uses significantly less RAM than using a queue, semaphore or event group to perform an equivalent operation. This is because each communication object (queue, semaphore or event group) must be created before it can be used, whereas enabling task notification functionality has a fixed overhead.

However, task notifications can't be used in the following scenarios:

1) Sending an event or data to an ISR

Task notifications can be used to send events and data from an ISR to a task, but they cannot be used to send events or data from a task to an ISR.

2) Enabling more than one receiving task

A communication object can be accessed by any task or ISR that knows its handle (which might be a queue handle, semaphore handle, or event group handle). Any number of tasks and ISRs can process events or data sent to any given communication object.

Task notifications are sent directly to the receiving task, so they can only be processed by the task to which the notification is sent. However, this is rarely a limitation in practical cases because, while it is common to have multiple tasks and ISRs sending to the same communication object, it is rare to have multiple tasks and ISRs receiving from the same communication object.

3) Buffering multiple data items

A queue can hold more than one data item at a time. Data that has been sent to the queue, but not yet received from the queue, is buffered inside the queue object. Task notifications send data to a task by updating the receiving task's notification value. A task's notification value can only hold one value at a time.

4) Broadcasting to more than one task

An event group can be used to send an event to more than one task at a time. Task notifications are sent directly to the receiving task, so can only be processed by the receiving task.

5) Waiting in the blocked state for a send to complete

If a communication object is temporarily in a state that means no more data or events can be written to it (for example, when a queue is full, no more data can be sent to the queue), then tasks attempting to write to the object can optionally enter the *Blocked* state to wait for their write operation to complete.

If a task attempts to send a task notification to a task that already has a notification pending, then it is not possible for the sending task to wait in the *Blocked* state for the receiving task to reset its notification state.

Low Power Support

FreeRTOS offers an easy way to tap into low power modes with the IDLE task hooks and tickless Idle mode.

It is common to reduce the power consumed by the microcontroller on which FreeRTOS is running by using the IDLE task hook to place the microcontroller into a low power state. The power saving that can be achieved by this method is limited by the necessity to periodically exit and then re-enter the low power state to process tick interrupts. Further, if the frequency of the tick interrupt is too high (wake from idle is too frequent), the energy and time consumed entering and then exiting a low power state for every tick will outweigh any potential power saving gains for all but the lightest power saving modes.

FreeRTOS supports a low power state that allows the microcontroller to periodically enter and exit low power consumption. The FreeRTOS tickless idle mode stops the periodic tick interrupt during idle periods (when there are no application tasks that are able to execute), which allows the microcontroller (MCU) to remain in a deep power saving state until either an interrupt occurs, or it is time for the RTOS kernel to transition a task into the *Ready* state. It then makes a correcting adjustment to the RTOS tick count value when the tick interrupt is restarted. The principle of the FreeRTOS tickless mode is to make the MCU enter the low-power mode to save system power consumption when the MCU is performing the idle task.

There are 3 types of sleep modes supported in FreeRTOS:

1) eAbortSleep: This mode denotes a task has been made ready, a context switch was pended or a tick interrupt has already occurred but was pended since the scheduler was suspended. It signals the RTOS to abort entering a sleep mode.

2) eStandardSleep: This mode allows to enter a sleep mode that will not last longer than the expected idle time.

3) eNoTasksWaitingTimeout: This mode is entered when no tasks are waiting for a timeout so it is safe to enter a sleep mode that can only be exited by an external interrupt or reset.

Conclusion

The original mission of the FreeRTOS project was to provide a free RTOS solution that was easy to use. That is, easy to build and deploy, on a Windows (or Linux) host computer, without having to figure out which source files are required, which include paths are required, or how to configure the real time debugging environment. This has been achieved through the provision of pre-configured, build-able, example projects for each officially support port.

In summary, it provides the following standard features, all of which have been deeply explored in this study:

- Preemptive or cooperative operation
- Optional time slicing
- Very flexible task priority assignment
- Flexible, fast, and light-weight task notification mechanisms
- Queues
- Binary Semaphores
- Counting Semaphores
- Mutexes
- Recursive Mutexes
- Software Timers
- Event Groups

The FreeRTOS MIT open source license is designed to ensure

- FreeRTOS can be used in commercial applications.
- FreeRTOS itself remains freely available to everybody.
- FreeRTOS users retain ownership of their intellectual property.

OpenRTOS is a commercially licensed version of FreeRTOS provided under license from Amazon Web Services by a third party.

SafeRTOS shares the same usage model as FreeRTOS, but has been developed in accordance with the practices, procedures, and processes necessary to claim compliance with various internationally recognized safety related standards.

The unprecedented global success of the FreeRTOS kernel comes from its compelling value proposition; FreeRTOS is professionally developed, strictly quality controlled, robust, supported, does not contain any intellectual property ownership ambiguity, and is truly free to use in commercial applications without any requirement to expose one’s proprietary source code.

Further, AWS's stewardship provides a global presence, expert security event response procedures, a large and diverse development team, expertise in formal verification, pen testing, memory safety proofs, and long-term support – all while maintaining FreeRTOS as a hardware, development tool, and cloud service-neutral open-source project. FreeRTOS development is transparent and community-driven in GitHub, and does not require any special tools or development practice.

FreeRTOS thrives on its simplicity. The whole OS can essentially be described as nothing more than a scheduler for threads, plus a TCP/IP stack.

But as microprocessors have become more powerful, the number of applications that require *hard real-time* scheduling continues to shrink. These applications do still exist, but they’re getting rarer. And that is why the *RTOS* is one of the least interesting things about FreeRTOS. It turns out that FreeRTOS is a great choice for a whole class of small devices, even though most probably don’t need a real-time OS.

References

What is RTOS?	https://www.spiceworks.com/tech/hardware/articles/what-is-rtos/
RTOS Characteristics	https://www.techtarget.com/searchdatacenter/definition/real-time-operating-system
RTOS vs GPOS	https://www.wellpcb.com/rtos-vs-gpos.html
Popular RTOS	https://www.lynx.com/embedded-systems-learning-center/most-popular-real-time-operating-systems-rtos
Amazon FreeRTOS	http://www.openrtos.net/FAQ_Amazon.html
History of FreeRTOS	http://www.openrtos.net/RTOS.html
FreeRTOS Documentation	https://www.freertos.org/Documentation/Mastering-the-FreeRTOS-Real-Time-Kernel.v1.0.pdf
Basic Guide of FreeRTOS	https://embeddedinventor.com/freertos-a-complete-beginners-guide/